



SAPIENZA
UNIVERSITÀ DI ROMA

Advanced Software Engineering

NNModelling

A Visual DSL for Neural Network Design

Authors

Davide Pietragalla 1965270

Luca Sforza 2050030

June 15, 2026

Contents

1	Introduction	3
2	Architecture	3
2.1	Metamodel-Driven Design	3
2.2	Three-Stage Compilation	4
2.3	Subflow Pattern	4
2.4	Join Model	4
2.5	Dynamic Execution	4
3	Conversion Pipeline	4
3.1	Custom Operations	5
4	Validation	6
4.1	Test Suite	6
4.2	DSL Expressiveness Validation	6
5	Conclusions	6

1 Introduction

NNModelling is a DSL (Domain-Specific Language) for designing neural network architectures using a visual diagrammatic language. Models drawn in the visual editor are automatically transpiled to working PyTorch Lightning code, enabling training and inference without manual implementation of the architecture.

The project follows a model-driven approach: the DSL syntax was first defined using a UML metamodel, then a front-end application was built to enable visual diagramming, and finally a code generation pipeline was implemented to produce executable configurations.

The system splits into two subsystems: a front-end visual editor built with Svelte 5, Svelte Flow, and TypeScript, where users drag, connect, and configure neural network modules; and a back-end code generation pipeline written in Python using PyTorch Lightning and Hydra/OmegaConf, which converts visual diagrams into Hydra configuration files and dynamically instantiates them into a working PyTorch model.

The core data pipeline is:

Visual Graph $\rightarrow$ NNTree (AST) $\rightarrow$ JSON $\rightarrow$ convert.py $\rightarrow$ Hydra YAML $\rightarrow$ LightningModule

2 Architecture

2.1 Metamodel-Driven Design

The entire system is driven by stereotype definitions stored as JSON files in the `Stereotypes/` directory. Each JSON file defines the Python class name for code generation (e.g., `nn.Linear`), visual properties such as color and dimensions for the canvas, typed parameters with defaults and display positions, and a category (`Layer`, `Join`, `Loss`, `Input`) that determines the node's structural behavior.

Stereotypes are loaded at compile time via Vite's `import.meta.glob`, scanning the `Stereotypes/**/*.json` directory structure automatically. Adding a new neural network layer requires only a new JSON file — no changes to the editor, compiler, or code generator.

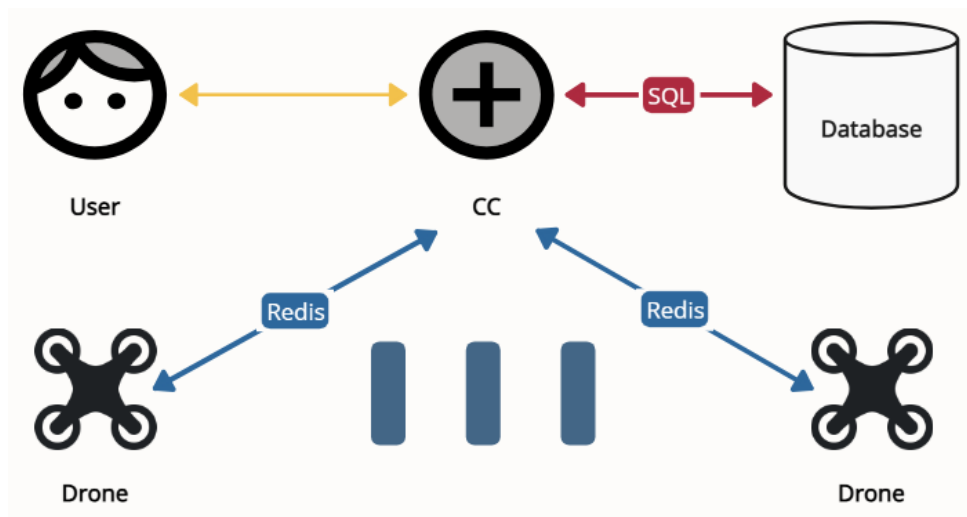


Figure 1: UML metamodel of the NNModelling DSL.

Currently the system supports 27 modules (Linear, Conv2d, ReLU, Tanh, Embedding, LayerNorm, Dropout, various loss functions, etc.), 6 joins (Addition, Concat, Einsum, MatMul,

ScaledDotProduct, MaskedScaledDotProduct), and 2 subflow behavioral templates (Repeat for sequential repetition and HorizontalRepeat for parallel copies via `vmap`).

2.2 Three-Stage Compilation

The visual graph undergoes three stages of transformation. First, the flat graph of nodes and edges is compiled into a hierarchical NNTree using Kahn’s topological sort, producing four node types: **sequential** chains compressed into a single node, **module** nodes with optional fan-out, **join** merge nodes with multiple inputs, and **subflow** containers with recursively compiled internal directed graphs. Second, the NNTree is serialized to JSON preserving all structural metadata including stereotype names, parameters, and subflow boundaries. Third, the Python script `convert.py` reads the NNTree JSON and generates 7 YAML configuration files covering the network architecture, optimizer, trainer, dataset, logging (Weights & Biases), and early stopping.

2.3 Subflow Pattern

Subflows are collapsible containers that group related nodes into reusable sub-architectures and can be nested recursively. Internally, each subflow uses a BFS topological traversal with in-degree counting for correct execution order.

Behavioral stereotypes extend subflow semantics: the **Repeat** stereotype duplicates the internal subgraph N times in sequence, similar to `nn.Sequential`; the **HorizontalRepeat** stereotype creates N parallel copies of the subgraph, executes them via `vmap` with independent weights, and concatenates outputs along the last dimension.

2.4 Join Model

Forks are implicit — any node output can connect to multiple targets without requiring a special node. Joins, conversely, must be explicit: they are special nodes that accept N inputs and produce a single output. Input ordering is preserved via `targetHandle` attributes on edges (e.g., `in-0`, `in-1`), which is critical for non-commutative operations like matrix multiplication and scaled dot product attention.

2.5 Dynamic Execution

The `Net` class, a `LightningModule`, dynamically builds its architecture from the Hydra configuration at runtime. A `ModuleDict` is populated by dispatching on node type (sequential, module, join, subflow), while the forward pass uses BFS traversal with in-degree tracking to ensure correct execution order through complex fork-join topologies. Join nodes collect all inputs ordered by `targetHandle` before executing, and loss functions are excluded from the execution graph and stored separately for metric selection.

3 Conversion Pipeline

The Python code generation pipeline handles the translation from DSL to executable model, as illustrated in the diagram below.

`convert.py` processes the NNTree JSON by parsing parameters via `ast.literal_eval` (which supports int, float, bool, list, dict, and None types), resolving `pythonClassName` to a Hydra `_target_` path (e.g., `nn.Linear` → `torch.nn.Linear`), and using `_recursive_: False` on subflow nodes to prevent Hydra from recursing into internal node maps. The output directory

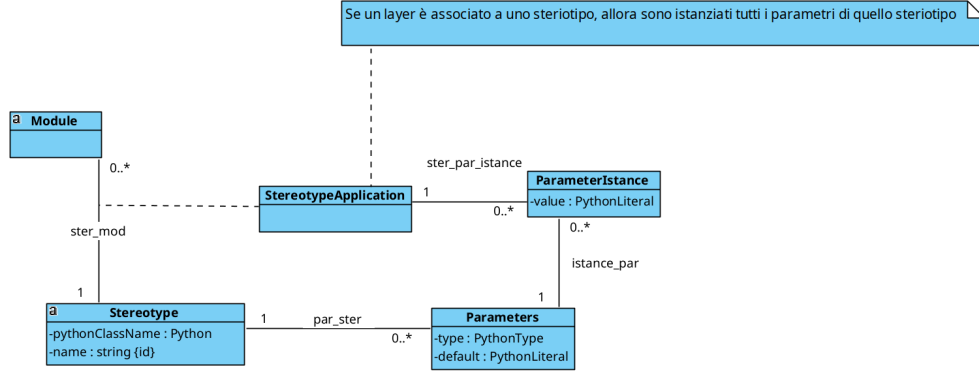


Figure 2: Transformer classifier designed in NNModelling. The diagram shows embedding, positional encoding, a 2-layer encoder with multi-head attention and residual connections, sequence pooling, and a linear classifier.

includes separate subdirectories for net, optimizer, trainer, dataset, wandb, and early stopping configurations.

3.1 Custom Operations

Eleven custom Python operations are implemented in `converted/src/ops/`, spanning three categories. Six join operations handle multi-input merges: Addition, Concat, Einsum, MatMul, ScaledDotProduct, and MaskedScaledDotProduct. Three subflow operations manage container execution: Subflow (BFS internal graph traversal), Repeat (N sequential copies), and HorizontalRepeat (N parallel copies via `vmap`). Two module operations provide PositionalEncoding (sinusoidal encoding) and SequencePool (mean over sequence dimension).

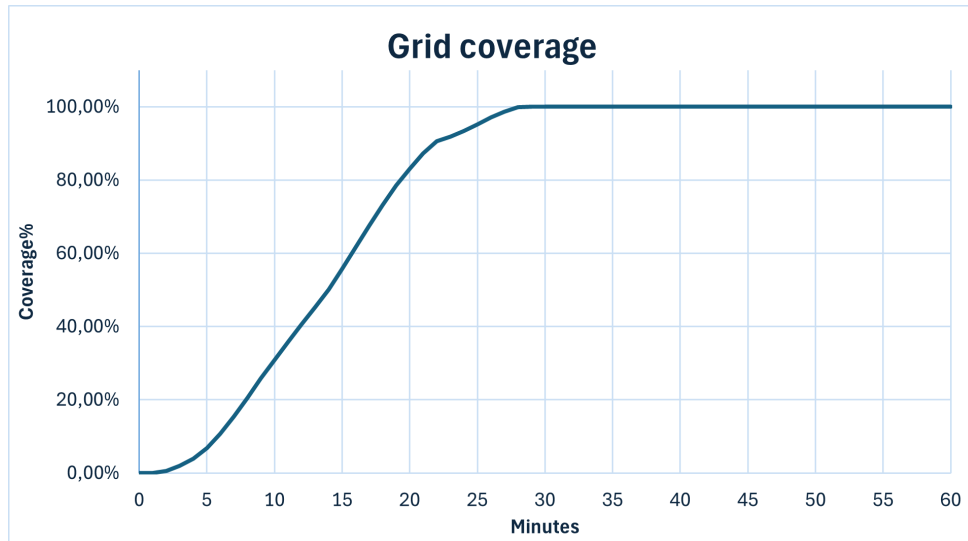


Figure 3: Training metrics for the transformer classifier on EnronSpam dataset. Double descent phenomenon is visible in validation loss.

4 Validation

4.1 Test Suite

The project includes 179 automated tests across both front-end and back-end. The front-end suite (76 tests, Vitest) covers NNTree compilation — including sequential chains, skip connections, joins, subflow boundary mapping, Repeat unrolling, nested subflows, and Fork passthrough — plus 10 connection validation tests. On the Python side, 38 pytest tests unit-test all 11 custom operations, 36 tests cover config generation (parameter parsing, layer config building, nested subflow config, and YAML structure verification across 5 diagram fixtures), 21 tests validate the dynamic model (in-degree computation, Net initialization, BFS forward traversal for various topologies), and 8 integration tests run the end-to-end pipeline from JSON fixture through Hydra configuration to Net instantiation and forward pass with real tensor shapes and gradient flow checks.

4.2 DSL Expressiveness Validation

To validate that NNModelling can express modern architectures, a full transformer classifier was built entirely within the visual DSL. The model includes token embedding (30,522 vocabulary $\times$ 128 dimensions), sinusoidal positional encoding, a 2-layer TransformerEncoder constructed via the Repeat stereotype, multi-head attention with 4 heads using HorizontalRepeat with Q/K/V projections, residual connections via explicit Fork nodes, and sequence pooling followed by a linear classifier. The diagram compiled successfully to a working PyTorch model that was trained on the EnronSpam dataset, confirming the DSL’s expressiveness for feedforward, convolutional, and encoder-only transformer architectures.

5 Conclusions

NNModelling demonstrates that a visual DSL can effectively bridge the gap between neural network architecture design and implementation. The system successfully compiles complex architectures including transformers with multi-head attention, residual connections, nested subflows, and sequential repetition.

Key achievements include a full end-to-end pipeline from visual design to trained model, an extensible stereotype system requiring no code changes for new layer types, support for complex architectural patterns (subflows, residual connections, parallel branches), and comprehensive test coverage (179 tests) across both front-end and back-end.

Several limitations remain. Training loop strategies such as GANs, contrastive learning, and adversarial training cannot be expressed in the DSL and must be implemented in Python outside the framework. The HorizontalRepeat join type is hardcoded to concatenation on the last dimension. The forward pass executes nodes sequentially in a single Python queue loop, even when the DAG contains independent fork-join branches that could run concurrently on separate CUDA streams — though HorizontalRepeat’s `vmap`-based execution is an exception, vectorizing N independent copies into a single batched GPU kernel and achieving true parallel execution at the subflow level. Additionally, there is no support for pre-trained models or transfer learning workflows.

Future work includes a configurable `join_type` parameter for HorizontalRepeat, a DAG parallel scheduler that replaces the single-threaded topological iteration with a concurrent CUDA stream dispatcher for arbitrary fork-join topologies, HuggingFace Spaces deployment for interactive browser demos, and a training server API for one-click training from the visual editor.

References

- [1] Svelte 5 Documentation. <https://svelte.dev/docs>
- [2] Svelte Flow Documentation. <https://svelteflow.dev/>
- [3] Paszke, A. et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” NeurIPS 2019.
- [4] Falcon, W. et al. “PyTorch Lightning.” <https://lightning.ai/>
- [5] Vaswani, A. et al. “Attention Is All You Need.” NeurIPS 2017.
- [6] Yadan, O. “Hydra - A Framework for Elegantly Configuring Complex Applications.” <https://hydra.cc/>